

ASSIGNMENT BRIEF

Module title	Programming Fundamentals
Module code	COM474
Module leader	Teri Birch
Assessment title	Coursework 4: Text-Based Adventure Game C++
Launch date	Thursday 2 nd October
Submission deadline	Thursday 7 th May 2026 16:30pm
Expected date for return of marks and feedback	<i>Grades and written feedback will be available on Moodle within 15 working days. This will consist of annotations on submission, and face to face dialogue if and when required.</i>
Module outcomes assessed	1. Identify syntax and structure of an industry-standard programming language. 2. Relate programming concepts to the context of subject area. 3. Apply programming techniques within an Integrated Development Environment. 4. Demonstrate design solutions to contextualised problems.
Assessment weighting	35%
Word count (if relevant)	N/A

Assessment task details - provide a description of the task

In this assignment, you will develop a text-based adventure game using the **C++** programming language. The player should be able to navigate through various rooms or scenarios and make decisions that influence the outcome of the game. The game must be logically structured, narrative-driven, and demonstrate your ability to apply core programming principles.

Your solution should be fully interactive in a console environment, providing descriptive outputs, handling user input, and managing game progression using structured programming and object-oriented techniques. ASCII art is required within the game to enhance player immersion.

To demonstrate design thinking, you must also **produce a Jackson Structured Programming (JSP) diagram before coding**. The JSP diagram should outline your game's structure, including navigation between rooms, combat, inventory, and different endings. A short **design rationale** (150–200 words) must be included in your documentation, explaining how your JSP diagram guided your program design.

Finally, you should provide a **reflection** (200–300 words) linking the programming concepts used in your game to your subject area (Computer Science, Cybersecurity, or Electrical Engineering), and explaining how the adventure game is a designed solution to a contextualised problem.

Task Requirements

The following features are required in your submission:

Game Structure and Navigation

- A minimum of six distinct rooms or scenarios must be included.
- Each scenario must include narrative description and at least two choices.
- Players must be able to navigate through these using clear typed input commands.
- The game must include at least two different endings, influenced by player decisions.

Game Logic and Flow

- Use of loops to manage ongoing gameplay until an ending is reached.
- Conditional logic must be used to handle choices, item use, and outcomes.

Prifysgol Wreccsam Wrexham University

- Include at least one combat encounter with a simple combat system. Players should be able to engage in combat using at least two tactical options (e.g., attack, flee, use item).

Inventory System

- Players must be able to collect, view, and use items.
- The inventory must influence game progression or outcomes (e.g., using a key to unlock a room, using an item during combat).

Code Structure

- Code must be modular, with appropriate use of functions for managing different aspects of the game (e.g., input handling, room descriptions, inventory, combat).
- The game must use object-oriented principles, and you are required to implement the following classes: **Player**, **Room**, **Item**, and **Enemy**. Each class should have appropriate attributes and methods relevant to its role in the game.

ASCII Art Integration

- ASCII art must be included within the game in at least three separate locations.
- The art must support the theme and narrative of the game (e.g., used at the title screen, during an encounter, or at a game ending).
- Students may create original ASCII art or source it from open-license libraries with appropriate attribution in the documentation.

Input and Output

- User input must be validated, and the program must respond clearly to invalid input.
- All text output must be clearly formatted for readability in a standard console or terminal.

Documentation (Required)

In addition to submitting your C++ code, you must submit a **documentation file** in Word format (.doc or .docx). This should include:

- A brief summary of the game and how to play it.
- A Jackson Structured Programming (JSP) diagram of your game structure.

- A design rationale (150–200 words) explaining how your JSP diagram shaped your implementation.
- An overview of the game's structure and code design (including description of classes and functions used).
- Screenshots showing:
 - The title or introduction screen
 - Room navigation
 - Inventory interaction
 - Combat functionality
 - At least two different game endings
 - Examples of ASCII art used in the game
- A reflection (200–300 words) linking your program to your subject area and explaining how it demonstrates a design solution to a contextualised problem.
- A complete copy of your C++ source code pasted into the document (not screenshots). This must appear in a clearly labelled section (e.g., *Appendix: Full Program Code*). The code must exactly match the .cpp file you submit separately.

Live Demo

You must attend a live demonstration session where you will:

- Run your game and demonstrate its features.
- Explain your program structure and logic.
- Present your JSP diagram and explain how it informed your program design.
- Answer questions about design, code, and relevance to your subject area.

Important:

Your work must be submitted in **two parts**:

1. Documentation as a **.doc or .docx file** to the Turnitin submission point.
This must include your design, rationale, reflection, screenshots, and the full program code pasted as text.
2. Program code as a single **.cpp file** to the code submission point.

PDFs, screenshots of code, or zipped folders will not be accepted.

Failure to submit **both** the documentation and the program file, or failure to attend the live demonstration, will result in an automatic fail for this assignment.

Generative Artificial Intelligence (GenAI) Guidance

Use of GenAI is NOT Permitted:

- **Code Development:** The use of GenAI tools (such as AI-driven code generators like ChatGPT, GitHub Copilot, or any similar AI-assisted coding tools) is **not permitted**. This assignment is intended to demonstrate your ability to design and code a text-based adventure game from scratch. Using GenAI tools would undermine the learning objectives of this assignment.
- **Documentation:** All aspects of the submitted Word document, including the code, screenshots, and explanations, must be created by you. GenAI tools must not be used to generate any part of the documentation as this is intended to reflect your individual understanding of the program you developed.

Important Warning:

- Be aware that information provided by AI tools can often be **out of date** or **incorrect**.

Consequences of Inappropriate Use:

- Any use of GenAI tools to generate content for the code or documentation will be considered **academic misconduct** and may result in a penalty, including the potential for your assignment to be graded as a **fail**.

Submission instructions - What should be the format of the submission? / Where should it be submitted?

You must submit your work in two parts:

1. Documentation File (.doc or .docx) submitted at the Turnitin submission point provided:
 - A Word document containing:
 - A brief summary of the game and how to play it.
 - A Jackson Structured Programming (JSP) diagram of your game structure.
 - A design rationale (150–200 words) explaining how your JSP diagram shaped your implementation.

- An overview of the game's structure and code design (including description of classes and functions used).
- Screenshots showing:
 1. The title or introduction screen
 2. Room navigation
 3. Inventory interaction
 4. Combat functionality
 5. At least two different game endings
 6. Examples of ASCII art used in the game
- A reflection (200–300 words) linking your program to your subject area and explaining how it demonstrates a design solution to a contextualised problem.
- A full copy of your program code pasted directly into the document as text (with syntax formatting). Screenshots of code are not accepted.

- File name format: StudentID-Coursework4-Documentation.docx

2. C++ Source Code File (.cpp)

- Submit your program file to the separate code submission point provided.
- The full program must be written in C++ and saved as a .cpp file.
- The code should be clearly structured and commented to explain the logic.
- File name format: StudentID-Coursework4.cpp

Live Demo (Compulsory)

You must attend a live demonstration session where you will:

3. Run your game and demonstrate its features.
4. Explain your program structure and logic.
5. Present your JSP diagram and explain how it informed your program design.
6. Answer questions about design, code, and relevance to your subject area.

Important:

Failure to submit the .cpp file, the documentation file, or to attend the live

demonstration will result in a **fail for this assignment**, regardless of the quality of any individual component.

Hints and tips

To help you succeed in this assignment, consider the following guidance:

Design and Planning

- Start by **mapping out your game world**. Define how rooms/scenarios are connected and what decisions are available to the player at each point.
- Develop a **narrative theme or storyline** early on. A consistent setting and plot (e.g., dungeon escape, space mission, detective mystery) help structure your design.
- Plan at least **two distinct endings** based on player choices (e.g., success/failure, heroic/cowardly, escape/capture).

Programming and Structure

- Break down the game into **functions** for specific tasks (e.g., `displayRoom()`, `handleInput()`, `combatEncounter()`).
- Implement at least one **class** such as Player, Room, Item, or Enemy to organise related data and behaviour.
- Ensure **inventory management** supports adding, removing, and checking items.
- Build a **simple combat system** that can be expanded (e.g., turn-based damage exchange or success chance).

User Interaction

- Provide **clear instructions and prompts** at every stage of the game.
- Use **validation for user input** to prevent unexpected behaviour or crashes.
- Keep output **readable** using line spacing, consistent formatting, and descriptive messages.

ASCII Art

- Use ASCII art to highlight key moments (e.g., title screen, a boss encounter, game endings).
- Avoid large or overly complex ASCII if it affects readability. Simplicity and thematic relevance are most effective.

Testing

- Play through your game **multiple times** to test different paths, especially:
 - Using or not using items
 - Winning and losing in combat
 - Invalid input scenarios
 - Triggering all possible endings

Marking and moderation

All grades/marks are indicative and may change when moderated – see the last page for the marking criteria.

Wrexham University Skills Framework Employability Skills Applied

On successful completion of this module, a student will have had opportunities to demonstrate achievement of the following Employability Skills;

Tick all that apply.

Adaptability & Flexibility (<i>Managing Change, Coping with Ambiguity, Transferable Skills</i>)	✓
Career Development (<i>Career information, Access to opportunity, Career readiness</i>)	
Critical Thinking (<i>Problem Solving, Analytical Skills, Critical Thinking</i>)	✓
Digital Capabilities (<i>Digital Tools & Software, Data Analysis, Digital Citizenship</i>)	✓
Enterprise and Entrepreneurship (<i>Innovation, Commercial Awareness and Influencing</i>)	
Interpersonal Skills (<i>Communication, Emotional Intelligence, Networking, Bilingualism</i>)	✓
Personal Skills (<i>Initiative, Leadership, Social intelligence</i>)	
Resilience (<i>Positive Mindset, Self Awareness, Motivation & Purpose</i>)	✓
Social Impact and Sustainability (<i>Social Responsibility, inclusivity, Cultural Awareness and application of the UN Sustainable development goals</i>)	

Marking criteria							
Mark (%)	0-39	40-49	50-59	60-69	70-79	80-89	90-100
Game Functionality and Logical Flow (20%) LO1, LO3	The program does not compile or run. Navigation is broken, rooms cannot be accessed, and no endings are available. The player cannot meaningfully interact with the game.	The program compiles but contains major flaws. Navigation is unstable, with broken or inaccessible rooms. Endings may be absent or unreachable. Player interaction is inconsistent and unreliable.	The game runs, and navigation between some rooms is possible, but bugs or missing links reduce clarity. Endings exist but are very limited, repetitive, or underdeveloped.	The game has a consistent logical flow. Navigation works in a predictable manner, endings are distinct and reachable, and only minor errors or inconsistencies remain.	The game is fully functional, with smooth navigation, working logic across rooms, and multiple distinct endings. Gameplay runs reliably, with no major issues.	The game is robust and engaging. All navigation and endings are implemented, logic is tested across different paths, and the flow is polished and professional.	The game demonstrates professional quality. All paths and endings are thoroughly tested and bug-free. Flow is seamless, with logical consistency and polished gameplay that anticipates user behaviour.
Story Design and Player Choice (15%) LO1, LO3	No coherent story is present. The player has little or no context, and choices are	A minimal story exists but is inconsistent or fragmented. Player choices are very limited and produce	A basic narrative is in place with some branching. Player choices exist but lack depth, often	The narrative is coherent and engaging. Player choices are meaningful and consistently	The story is creative and offers multiple meaningful paths. Decisions clearly shape	The narrative is immersive and well-crafted. Choices are smoothly integrated into the story and	The narrative is excellent, demonstrating professional-quality storytelling. Player

	absent, irrelevant, or meaningless.	little variation in outcomes.	leading to repetitive or predictable outcomes.	lead to different, distinct outcomes.	the story direction and endings, enhancing replayability.	affect outcomes significantly, showing thoughtful design.	decisions fundamentally alter progression and endings, creating a rich, branching experience with high engagement.
Code Structure and OOP Implementation (15%) LO1, LO3	Code is incomplete, unstructured, and very difficult to read. No modularity is present. Variables are unclear, and the required classes (Player, Room, Item, Enemy) are not implemented	Minimal code structure is present. A few functions exist but are inconsistently applied. Attempt is made to use classes, but not all four required (Player, Room, Item, Enemy) are implemented, or implementation	Code shows some organisation. Functions are used for certain tasks, and all four required classes exist but with limited effectiveness (e.g., missing methods, poor attribute choices). Variable names are partly clear,	Code is logically structured, with modular functions and all four required classes implemented in a basic but functional way. Classes interact with each other, supporting program readability.	Code is well-structured with multiple reusable functions and well-implemented classes (Player, Room, Item, Enemy) that interact appropriately. Naming is consistent, and comments are detailed	Code is highly modular and efficient. All four classes are well-designed, reusable, and documented. Interactions between them are logical and efficient. Naming and comments demonstrate clear planning and	Code is professional and demonstrates advanced OOP principles. The required classes are fully integrated, modular, scalable, and extensible. Advanced OOP features are applied where appropriate.

Prifysgol Wreccsam Wrexham University

	at all. No meaningful use of OOP.	s are very weak. Naming conventions and comments are unclear.	and a few comments exist.	Naming is consistent, and comments explain most program logic.	enough to aid maintainability.	professionalism.	Code is fully documented with insightful comments, demonstrating best practice.
Combat and Inventory System (15%) LO1, LO3	Combat and inventory are missing or completely non-functional. Items cannot be used, and combat plays no role in gameplay.	A very basic combat or inventory system exists but is unreliable. Items have little purpose, and combat has minimal effect on outcomes.	Combat and inventory are functional but simplistic. Items and combat choices exist but affect gameplay only in limited ways.	Combat and inventory are clearly implemented. Items can be collected and used, and combat influences progression meaningfully.	Combat and inventory are polished and varied. Different item types exist, and tactical combat choices create different outcomes.	Combat and inventory are robust, balanced, and integrated into the story. Systems add depth and variety, enhancing gameplay significantly.	Combat and inventory systems are exemplary. They are seamless, strategic, and deeply integrated into both gameplay and story. They provide professional-level complexity and immersion.
Input Handling and User Interaction	Input handling is absent. The game crashes on invalid entries, and	Minimal input validation is attempted. Invalid entries frequently	Input validation works in some cases but is inconsistent. Prompts are	Input validation is reliable. Prompts are clear, and	Input handling is strong. Prompts are user-friendly, error	Input handling is polished and anticipates most errors. Prompts are	Input handling is excellent. All potential errors are anticipated and handled

Prifysgol Wreccsam Wrexham University

(10%) LO1, LO3	prompts are missing, unclear, or misleading. The player cannot progress reliably.	cause errors or unpredictable behaviour. Prompts are present but confusing or poorly worded.	clear enough to allow play but lack polish. Error handling is minimal or unhelpful.	errors are handled gracefully, preventing crashes	messages are clear, and interaction supports smooth gameplay.	professional, and feedback is precise, giving players confidence in their choices.	gracefully. Prompts and feedback are clear, professional, and fully support a guided, user-centred experience.
ASCII Art Integration (5%) LO1, LO3	ASCII art is absent or irrelevant. No attempt has been made to integrate visuals into the game.	ASCII art is present but minimal, poorly drawn, or out of place. It adds little or no value to the game experience.	ASCII art is included in some places, adding minor atmosphere, but it is inconsistent, basic, or repetitive.	ASCII art is placed effectively and enhances immersion in important parts of the game.	ASCII art is creative and contributes significantly to the game's theme and atmosphere.	ASCII art is polished and used consistently to enhance immersion and storytelling.	ASCII art is excellent. It is fully integrated, professional in quality, and elevates the game to a highly immersive, engaging experience.
Documentation and Reflection (5%) LO1, LO2, LO3, LO4	Documentation is absent. No screenshots, rationale, or reflection are provided..	Documentation is minimal and poorly presented. Screenshots are incomplete,	Documentation includes basic screenshots and explanation. A rationale and	Documentation is complete, with screenshots, rationale, and reflection that	Documentation is strong. Screenshots are accurate, the rationale is coherent, and	Documentation is detailed and reflective. The rationale is thoughtful, and the reflection is	Documentation is professional, comprehensive, and critically reflective. Screenshots,

		and rationale or reflection is missing or irrelevant.	reflection are present but are very superficial or generic.	link to program structure.	the reflection links programming concepts to the subject area.	evaluative, showing awareness of subject relevance.	rationale, and reflection are fully integrated. The reflection is original, discipline-specific, and demonstrates critical insight.
Live Demonstration & Explanation (5%) LO1-LO4	The student does not attend or cannot run the program. No explanation or understanding is shown.	The student attends but struggles to demonstrate the program. Explanations are unclear, and most questions cannot be answered.	The student demonstrates some program features and provides very basic explanations. Answers to questions are hesitant and show gaps in understanding.	The student clearly demonstrates the program and JSP diagram. Explanations are adequate, and most questions are answered satisfactorily.	The student is confident in demonstrating the program. Explanations of logic and design are clear, and answers to questions are accurate.	The student demonstrates strong understanding. Explanations link code, design, and subject area. Answers are thoughtful and reflective.	The student delivers an outstanding demonstration. Explanations are fluent and critical, covering code, JSP design, and subject relevance. Questions are answered with confidence and insight.